Relatório Técnico: Redes Definidas por Software (SDN)

Descoberta de Topologias de Rede com Mininet, Ryu e NetworkX

Autores:

Uálaci dos Anjos Júnior

Jefferson Johnson

19 de outubro de 2026

1 Introdução

Repositório do Projeto no GitHub: https://github.com/vivendo88/Redes---Trabalho-SDN-1

Este relatório apresenta o desenvolvimento, a fundamentação teórica e os resultados experimentais do trabalho de **Descoberta de Topologias de Rede com Mininet e Ryu**. O objetivo principal consiste em instanciar topologias arbitrárias no Mininet a partir de arquivos de texto em formato de lista de arestas e realizar o mapeamento dinâmico completo da rede, identificando switches, enlaces entre switches e hosts conectados, por meio de um controlador SDN desenvolvido com o framework Ryu utilizando o protocolo OpenFlow 1.3 e a biblioteca NetworkX.

2 Arquitetura da Solução e Implementação

A solução foi dividida em dois módulos principais: o script de instanciação da topologia no Mininet (`topologia.py`) e a aplicação do controlador SDN no Ryu (`topo_discover.py`).

2.1 Leitura e Instanciação da Topologia no Mininet (`topologia.py`)

O script `topologia.py` recebe como argumento de linha de comando o caminho para um arquivo de texto contendo a lista de arestas entre switches (como `topo_triangle.txt` ou `topo_someloops.txt`). O funcionamento segue as etapas abaixo:

1. **Configuração OpenFlow 1.3 e Controlador Remoto:** O script configura a classe `OVSKernelSwitch` com o parâmetro `protocols='OpenFlow13'` (por meio de `functools.partial`) para garantir que todos os switches operem na versão 1.3 do protocolo OpenFlow, conectando-se a um `RemoteController` na porta `6633`.

2. **Parsing Dinâmico e Criação de Nós:** Ao ler cada linha do arquivo de arestas (`s_orig s_dest`), o script verifica se cada switch já foi instanciado no dicionário `switches`. Caso ainda não exista, o switch é adicionado via `net.addSwitch(s_name)` e, imediatamente, é criado um host correspondente (`h1` para `s1`, `h2` para `s2`, etc.) conectado à primeira porta disponível daquele switch (**porta 1**). Em seguida, é criado o enlace entre os dois switches da aresta com `net.addLink()`.

3. **Anúncio de Hosts via Gratuitous ARP:** Após iniciar a rede com `net.start()` e aguardar 3 segundos para a convergência inicial do controlador, cada host executa o disparo de um pacote *Gratuitous ARP* na sua interface ativa:

```
for h in net.hosts:
    iface = h.defaultIntf().name
    h.cmd(f'arping -U -c 1 -I {iface} {h.IP()}')
    sleep(0.2)
```

Listing 1: Trecho de instanciacao e disparo do Gratuitous ARP em topologia.py

2.2 Funcionamento da Flag `--observe-links` e das Funções `get_switch` / `get_link`

Em uma rede SDN pura sem protocolos tradicionais de camada 2 pré-configurados nos switches, o controlador não conhece previamente quais switches estão interconectados entre si. Para resolver isso no Ryu, utiliza-se o parâmetro `--observe-links` na inicialização do controlador:

```
$ ryu-manager --observe-links topo_discover.py
```

Como funciona internamente:

1. **Injeção de Pacotes LLDP (*Link Layer Discovery Protocol*):** A flag `--observe-links` ativa automaticamente o módulo interno `ryu.topology.switches`. Quando um switch se conecta ao controlador, esse módulo instala uma regra de fluxo para capturar quadros LLDP e passa a enviar periodicamente mensagens OpenFlow `PacketOut` instruindo cada switch a transmitir quadros LLDP por todas as suas portas ativas. Cada quadro LLDP carrega o identificador do switch de origem (`DPID`) e o número da porta de saída (`port_no`).

2. **Descoberta de Enlaces entre Switches:** Quando um switch vizinho recebe esse quadro LLDP em uma de suas portas, ele o encaminha ao controlador via mensagem `PacketIn`. Ao analisar o quadro, o módulo de topologia do Ryu identifica que o pacote saiu da porta de origem `(src_dpid,src_port)` e chegou na porta de destino `(dst_dpid,dst_port)`, registrando um enlace direcional entre os dois switches e disparando o evento `EventLinkAdd`. Como ambos os switches enviam LLDP, cada cabo físico entre dois switches resulta em **dois enlaces direcionais** mapeados (`sA -> sB` e `sB -> sA`).

3. **Extração via `get_switch` e `get_link`:** Na aplicação `topo_discover.py`, os decorators dos eventos `EventSwitchEnter` e `EventLinkAdd` interceptam a entrada de novos switches e a detecção de novos enlaces. Dentro desses métodos, as funções `get_switch()` e `get_link()` da API `ryu.topology.api` consultam o estado mantido pelo módulo de topologia do Ryu, retornando os objetos atualizados de switches (`switch.dp.id`) e enlaces (`src.dpid`, `dst.dpid`, `src.port_no` e `dst.port_no`), que são armazenados nas listas `self.switches` e `self.links`.

```
@set_ev_cls(event.EventSwitchEnter)
def _event_switch_enter_handler(self, ev):
    switch_list = get_switch(self.topology_api_app, None)
    self.switches = [switch.dp.id for switch in switch_list]
    self.logger.info("Switches mapeados atualmente: %s", self.switches)

@set_ev_cls(event.EventLinkAdd)
def _event_link_add_handler(self, ev):
    link_list = get_link(self.topology_api_app, None)
    self.links = [(link.src.dpid, link.dst.dpid, link.src.port_no, link.
       dst.port_no) for link in link_list]
    self.logger.info("Total de enlaces descobertos: %d", len(self.links)
       )
```

Listing 2: Captura de switches e enlaces via API de Topologia do Ryu em topo_discover.py

2.3 Interceptação via `PacketIn` e Descoberta de Hosts com Gratuitous ARP

Enquanto os switches são descobertos no handshake OpenFlow e os enlaces entre switches são mapeados via LLDP, os hosts finais não participam do protocolo LLDP. Além disso, como as topologias testadas possuem ciclos (*loops*), o uso de *flooding* convencional sem árvore geradora causaria tempestades de broadcast (*broadcast storms*).

Para descobrir os hosts de forma limpa e controlada, o processo ocorre em três passos:

1. **Instalação da Regra *Table-Miss* (OpenFlow 1.3):** No OpenFlow 1.3, switches recém-conectados descartam por padrão pacotes que não casam com nenhuma regra. Por isso, o método `switch_features_handler`, acionado no evento `EventOFPSwitchFeatures` durante a fase `CONFIG_DISPATCHER`, instala uma regra de prioridade `0` com `OFPMatch()` vazio e ação de envio para a porta `OFPP_CONTROLLER`. Isso instrui o switch a encapsular qualquer pacote desconhecido em uma mensagem `PacketIn` e enviá-la ao controlador.

2. **Recepção da Mensagem `PacketIn`:** Quando o script do Mininet executa `arping -U -c 1` em cada host, o host emite um pacote ARP não solicitado (*Gratuitous ARP*) anunciando seu próprio endereço IP e MAC. Ao chegar na porta do switch à qual o host está conectado, o pacote atinge a regra *Table-Miss* e gera um evento `EventOFPPacketIn` no controlador, sem ser inundado na rede.

3. **Extração e Armazenamento dos Hosts:** O handler `_packet_in_handler` extrai dos metadados da mensagem OpenFlow o identificador do switch (`datapath.id`) e a porta de entrada (`in_port`). Em seguida, utiliza a biblioteca `ryu.lib.packet` para decodificar o quadro Ethernet e verificar a presença do protocolo ARP com `pkt.get_protocol(arp.arp)`. Sendo um pacote ARP, o controlador extrai o IP de origem (`arp_pkt.src_ip`) e o MAC de origem (`arp_pkt.src_mac`), registrando o host no dicionário `self.hosts`:

```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def _packet_in_handler(self, ev):
    msg = ev.msg
    datapath = msg.datapath
    in_port = msg.match['in_port']

    pkt = packet.Packet(msg.data)
    arp_pkt = pkt.get_protocol(arp.arp)

    if arp_pkt:
        self.logger.info("ARP recebido do IP %s, MAC %s no Switch %s,
            porta %s",
                         arp_pkt.src_ip, arp_pkt.src_mac, datapath.id,
                            in_port)

        self.hosts[arp_pkt.src_ip] = {
            'mac': arp_pkt.src_mac,
            'dpid': datapath.id,
            'port': in_port
        }

        self.print_full_topology()
```

```
        self.salvar_grafo_descoberto()
```

Listing 3: Tratamento do PacketIn e registro de hosts via ARP em topo_discover.py

2.4 Geração Gráfica com NetworkX (`salvar_grafo_descoberto`)

A cada novo host descoberto, além de imprimir o relatório textual formatado no console (`print_full_topology()`), o controlador reconstrói um objeto `nx.Graph()` da biblioteca **NetworkX**. Os switches são inseridos como nós azuis (`lightblue`), os enlaces descobertos via LLDP formam as arestas entre switches, e os hosts armazenados em `self.hosts` são adicionados como nós verdes (`lightgreen`) conectados aos seus respectivos switches, exportando a figura para arquivo PNG com auxílio do `matplotlib`.

3 Validação Experimental e Logs de Execução

Os testes foram conduzidos nas duas topologias especificadas: **Triangle** (`topo_triangle.txt`) e **Someloops** (`topo_someloops.txt`).

3.1 Experimento 1: Topologia *Triangle* (`topo_triangle.txt`)

O arquivo `topo_triangle.txt` define três switches interconectados em formato de anel triangular:

```
s1 s2
s2 s3
s1 s3
```

Listing 4: Conteudo do arquivo topo_triangle.txt

3.1.1 Log de Execução no Terminal do Mininet

```
$ sudo python3 topologia.py topo_triangle.txt
[sudo: authenticate] Password:
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 3 switches
s1 s2 s3 ...
Aguardando inicialização do controlador...
Enviando Gratuitous ARP de todos os hosts para descoberta pelo Ryu...
*** Starting CLI:
mininet>
```

Listing 5: Saida do terminal do Mininet para a topologia Triangle

3.1.2 Log de Execução no Console do Controlador Ryu

```
$ ryu-manager --observe-links topo_discover.py
loading app topo_discover.py
generated new fontManager
```

```
loading app ryu.topology.switches
loading app ryu.controller.ofp_handler
instantiating app topo_discover.py of TopoDiscover
instantiating app ryu.topology.switches of Switches
instantiating app ryu.controller.ofp_handler of OFPHandler
Switches mapeados atualmente: [1]
Switches mapeados atualmente: [1, 2]
Total de enlaces descobertos: 1
Total de enlaces descobertos: 2
Switches mapeados atualmente: [1, 2, 3]
Total de enlaces descobertos: 4
Total de enlaces descobertos: 4
Total de enlaces descobertos: 6
Total de enlaces descobertos: 6
ARP recebido do IP 10.0.0.1, MAC 6a:1b:18:ec:78:9e no Switch 1, porta 1
...
ARP recebido do IP 10.0.0.2, MAC 92:82:cd:c3:58:44 no Switch 2, porta 1
...
ARP recebido do IP 10.0.0.3, MAC 7a:54:57:b7:71:51 no Switch 3, porta 1

==================================================
           TOPOLOGIA COMPLETA MAPEADA
==================================================
[SWITCHES ATIVOS]
 -> Switch DPID: 1 (s1)
 -> Switch DPID: 2 (s2)
 -> Switch DPID: 3 (s3)

[ENLACES ENTRE SWITCHES]
 -> Switch 2 (porta 2) <---> Switch 1 (porta 2)
 -> Switch 1 (porta 2) <---> Switch 2 (porta 2)
 -> Switch 3 (porta 3) <---> Switch 1 (porta 3)
 -> Switch 3 (porta 2) <---> Switch 2 (porta 3)
 -> Switch 1 (porta 3) <---> Switch 3 (porta 3)
 -> Switch 2 (porta 3) <---> Switch 3 (porta 2)

[HOSTS CONECTADOS]
 -> IP: 10.0.0.1 | MAC: 6a:1b:18:ec:78:9e | Conectado ao Switch: 1 (
    porta 1)
 -> IP: 10.0.0.2 | MAC: 92:82:cd:c3:58:44 | Conectado ao Switch: 2 (
    porta 1)
 -> IP: 10.0.0.3 | MAC: 7a:54:57:b7:71:51 | Conectado ao Switch: 3 (
    porta 1)
==================================================

Imagem do grafo salva em: topologia_descoberta.png
```

Listing 6: Saida completa do console do Ryu para a topologia Triangle

3.1.3 Evidências de Terminal e Grafo NetworkX (*Triangle*)

A Figura 1 apresenta a execução do Mininet (Terminal 2) instanciando os 3 switches e 3 hosts da topologia *Triangle*, enquanto a Figura 2 exibe a saída do console do controlador Ryu (Terminal 1) com a topologia mapeada. A Figura 3 apresenta o grafo gerado automaticamente pelo NetworkX para a topologia *Triangle*.

```
(.venv) ualaci@DESKTOP-H8I7KK1:~/win/Documents/Git/Redes---Trabalho-SDN-1$ sudo python3 topologia.py topo_
triangle.txt
Unable to contact the remote controller at 127.0.0.1:6633
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 3 switches
s1 s2 s3 ...
Aguardando inicialização do controlador...
Enviando Gratuitous ARP de todos os hosts para descoberta pelo Ryu...


*** Starting CLI:
mininet>
```

Figura 1: Terminal 2 (Mininet) instanciando a topologia *Triangle* e disparando Gratuitous ARP.

```
^C(.venv) ualaci@DESKTOP-H8I7KK1:~/win/Documents/Git/Redes---Trabalho-SDN-1$ ryu-manager --observe-links t
opo_discover.py
loading app topo_discover.py
loading app ryu.topology.switches
loading app ryu.controller.ofp_handler
instantiating app topo_discover.py of TopoDiscover
instantiating app ryu.topology.switches of Switches
instantiating app ryu.controller.ofp_handler of OFPHandler
Switches mapeados atualmente: [1, 3]
Switches mapeados atualmente: [1, 3, 2]
Switches mapeados atualmente: [1, 3, 2]
Total de enlaces descobertos: 4
Total de enlaces descobertos: 6
Total de enlaces descobertos: 6
Total de enlaces descobertos: 6
Total de enlaces descobertos: 6
Total de enlaces descobertos: 6
ARP recebido do IP 10.0.0.1, MAC fe:00:dd:25:99:cf no Switch 1, porta 1

==================================================
           TOPOLOGIA COMPLETA MAPEADA
==================================================
[SWITCHES ATIVOS]
 -> Switch DPID: 1 (s1)
 -> Switch DPID: 2 (s2)
 -> Switch DPID: 3 (s3)

[ENLACES ENTRE SWITCHES]
 -> Switch 1 (porta 2) <---> Switch 2 (porta 2)
 -> Switch 1 (porta 3) <---> Switch 3 (porta 3)
 -> Switch 3 (porta 2) <---> Switch 2 (porta 3)
 -> Switch 3 (porta 3) <---> Switch 1 (porta 3)
 -> Switch 2 (porta 2) <---> Switch 1 (porta 2)
 -> Switch 2 (porta 3) <---> Switch 3 (porta 2)

[HOSTS CONECTADOS]
 -> IP: 10.0.0.1 | MAC: fe:00:dd:25:99:cf | Conectado ao Switch: 1 (porta 1)
==================================================

Imagem do grafo salva em: topologia_triangulo_descoberta.png
```

Figura 2: Terminal 1 (Controlador Ryu) exibindo a topologia *Triangle* mapeada.

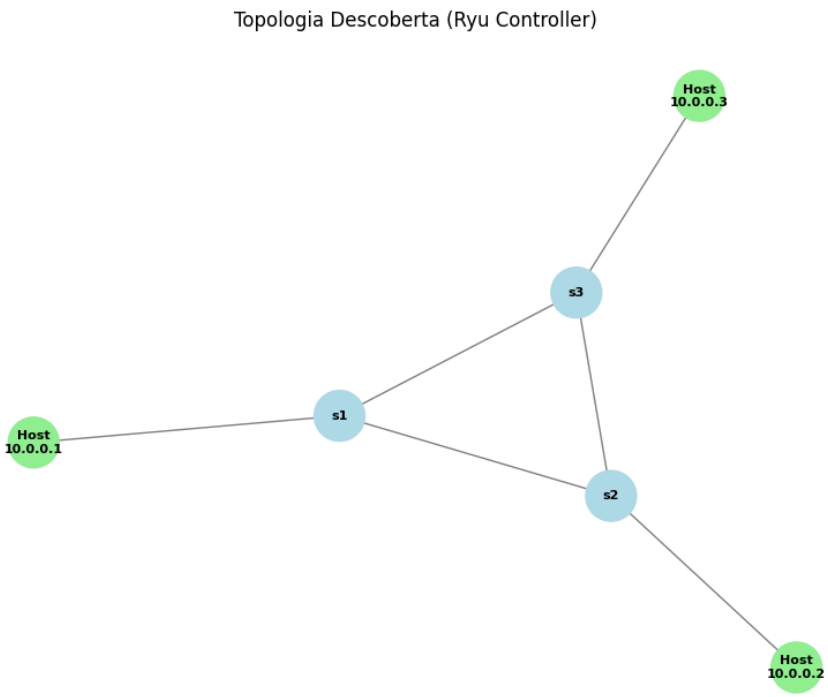


Figura 3: Grafo da topologia *Triangle* gerado com NetworkX a partir dos dados descobertos pelo Ryu.

Análise de Consistência (*Triangle*): Verifica-se total consistência entre a rede instanciada pelo Mininet e a topologia descoberta pelo Ryu:

- Os 3 switches (`s1`, `s2`, `s3`) foram registrados com DPIDs 1, 2 e 3.
- As 3 arestas do arquivo resultaram em 6 enlaces direcionais LLDP: `s1(p2) <-> s2(p2)`, `s2(p3) <-> s3(p2)` e `s1(p3) <-> s3(p3)`.
- Como cada host é vinculado ao seu switch imediatamente na criação do switch, todos os hosts (`10.0.0.1`, `10.0.0.2` e `10.0.0.3`) ocupam a **porta 1** de seus respectivos switches (`s1`, `s2` e `s3`).

3.2 Experimento 2: Topologia *Someloops* (`topo_someloops.txt`)

O arquivo `topo_someloops.txt` define uma malha com múltiplos caminhos e ciclos composta por 6 switches (`s1` a `s6`) e 8 arestas entre switches:

```
s1 s2
s1 s4
s1 s5
s2 s4
s2 s6
s3 s4
s3 s5
s4 s6
```

Listing 7: Conteudo do arquivo topo_someloops.txt

3.2.1 Logs de Execução (*Someloops*)

Ao executar `sudo python3 topologia.py topo_someloops.txt`, o Mininet instancia os 6 switches e seus 6 hosts correspondentes na ordem em que os nós aparecem na lista de arestas (`s1`, `s2`, `s4`, `s5`, `s6`, `s3`):

```
$ sudo python3 topologia.py topo_someloops.txt
*** Configuring hosts
h1 h2 h4 h5 h6 h3
*** Starting controller
c0
*** Starting 6 switches
s1 s2 s4 s5 s6 s3 ...
Aguardando inicialização do controlador...
Enviando Gratuitous ARP de todos os hosts para descoberta pelo Ryu...
*** Starting CLI:
mininet>
```

Listing 8: Saida do terminal do Mininet para a topologia Someloops

No console do controlador Ryu, após a descoberta dos 6 switches e dos 16 enlaces direcionais (correspondentes às 8 arestas bidirecionais), bem como a recepção dos 6 pacotes Gratuitous ARP, a topologia completa mapeada é impressa:

```
================================================
           TOPOLOGIA COMPLETA MAPEADA
================================================
[SWITCHES ATIVOS]
 -> Switch DPID: 1 (s1)
 -> Switch DPID: 2 (s2)
 -> Switch DPID: 3 (s3)
 -> Switch DPID: 4 (s4)
 -> Switch DPID: 5 (s5)
 -> Switch DPID: 6 (s6)

[ENLACES ENTRE SWITCHES]
 -> Switch 2 (porta 2) <---> Switch 1 (porta 2)
 -> Switch 1 (porta 2) <---> Switch 2 (porta 2)
 -> Switch 4 (porta 2) <---> Switch 1 (porta 3)
 -> Switch 4 (porta 3) <---> Switch 2 (porta 3)
 -> Switch 1 (porta 3) <---> Switch 4 (porta 2)
 -> Switch 2 (porta 3) <---> Switch 4 (porta 3)
 -> Switch 5 (porta 2) <---> Switch 1 (porta 4)
 -> Switch 1 (porta 4) <---> Switch 5 (porta 2)
 -> Switch 6 (porta 3) <---> Switch 4 (porta 5)
 -> Switch 6 (porta 2) <---> Switch 2 (porta 4)
 -> Switch 4 (porta 5) <---> Switch 6 (porta 3)
 -> Switch 2 (porta 4) <---> Switch 6 (porta 2)
 -> Switch 3 (porta 2) <---> Switch 4 (porta 4)
 -> Switch 3 (porta 3) <---> Switch 5 (porta 3)
 -> Switch 4 (porta 4) <---> Switch 3 (porta 2)
 -> Switch 5 (porta 3) <---> Switch 3 (porta 3)

[HOSTS CONECTADOS]
 -> IP: 10.0.0.1 | MAC: 0e:61:b3:62:48:f5 | Conectado ao Switch: 1 (
    porta 1)
 -> IP: 10.0.0.2 | MAC: 76:a7:f8:df:8d:42 | Conectado ao Switch: 2 (
    porta 1)
```

```
 -> IP: 10.0.0.3 | MAC: 8e:77:da:c0:1c:c7 | Conectado ao Switch: 4 (
    porta 1)
 -> IP: 10.0.0.4 | MAC: 6a:5d:28:6e:26:15 | Conectado ao Switch: 5 (
    porta 1)
 -> IP: 10.0.0.6 | MAC: d6:e3:19:9b:03:2e | Conectado ao Switch: 3 (
    porta 1)
 -> IP: 10.0.0.5 | MAC: 9a:75:39:cc:f9:71 | Conectado ao Switch: 6 (
    porta 1)
==================================================

Imagem do grafo salva em: topologia_descoberta.png
```

Listing 9: Saida consolidada do controlador Ryu para a topologia Someloops

3.2.2 Evidências de Terminal e Grafo NetworkX (*Someloops*)

A Figura 4 apresenta a execução do Mininet (Terminal 2) instanciando os 6 switches e 6 hosts da topologia *Someloops*, enquanto a Figura 5 exibe a saída do console do controlador Ryu (Terminal 1). A Figura 6 exibe a representação gráfica gerada pelo NetworkX para a topologia *Someloops*.

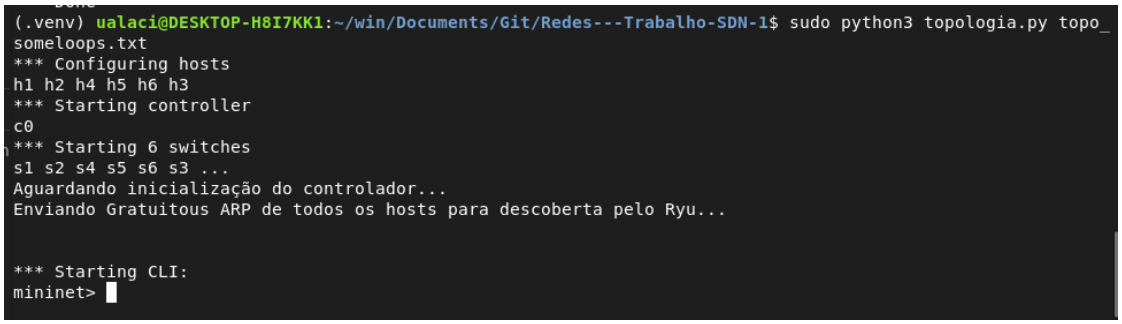


Figura 4: Terminal 2 (Mininet) instanciando a topologia *Someloops* e disparando Gratuitous ARP.

```
================================================
          TOPOLOGIA COMPLETA MAPEADA
================================================
[SWITCHES ATIVOS]
 -> Switch DPID: 1 (s1)
 -> Switch DPID: 2 (s2)
 -> Switch DPID: 3 (s3)
 -> Switch DPID: 4 (s4)
 -> Switch DPID: 5 (s5)
 -> Switch DPID: 6 (s6)

[ENLACES ENTRE SWITCHES]
 -> Switch 2 (porta 2) <---> Switch 1 (porta 2)
 -> Switch 1 (porta 2) <---> Switch 2 (porta 2)
 -> Switch 4 (porta 2) <---> Switch 1 (porta 3)
 -> Switch 4 (porta 3) <---> Switch 2 (porta 3)
 -> Switch 1 (porta 3) <---> Switch 4 (porta 2)
 -> Switch 2 (porta 3) <---> Switch 4 (porta 3)
 -> Switch 5 (porta 2) <---> Switch 1 (porta 4)
 -> Switch 1 (porta 4) <---> Switch 5 (porta 2)
 -> Switch 6 (porta 3) <---> Switch 4 (porta 5)
 -> Switch 6 (porta 2) <---> Switch 2 (porta 4)
 -> Switch 4 (porta 5) <---> Switch 6 (porta 3)
 -> Switch 2 (porta 4) <---> Switch 6 (porta 2)
 -> Switch 3 (porta 2) <---> Switch 4 (porta 4)
 -> Switch 3 (porta 3) <---> Switch 5 (porta 3)
 -> Switch 4 (porta 4) <---> Switch 3 (porta 2)
 -> Switch 5 (porta 3) <---> Switch 3 (porta 3)

[HOSTS CONECTADOS]
 -> IP: 10.0.0.1 | MAC: 32:c5:5c:ab:68:b8 | Conectado ao Switch: 1 (porta 1)
 -> IP: 10.0.0.2 | MAC: da:b1:5c:12:1c:91 | Conectado ao Switch: 2 (porta 1)
 -> IP: 10.0.0.3 | MAC: aa:d4:84:8b:a2:eb | Conectado ao Switch: 4 (porta 1)
 -> IP: 10.0.0.4 | MAC: 6e:00:79:de:fa:0e | Conectado ao Switch: 5 (porta 1)
 -> IP: 10.0.0.5 | MAC: b2:36:a2:fb:a3:ba | Conectado ao Switch: 6 (porta 1)
 -> IP: 10.0.0.6 | MAC: fe:ee:5b:26:c3:8b | Conectado ao Switch: 3 (porta 1)
================================================

Imagem do grafo salva em: topologia_someloops_descoberta.png
```

Figura 5: Terminal 1 (Controlador Ryu) exibindo a topologia *Someloops* mapeada.

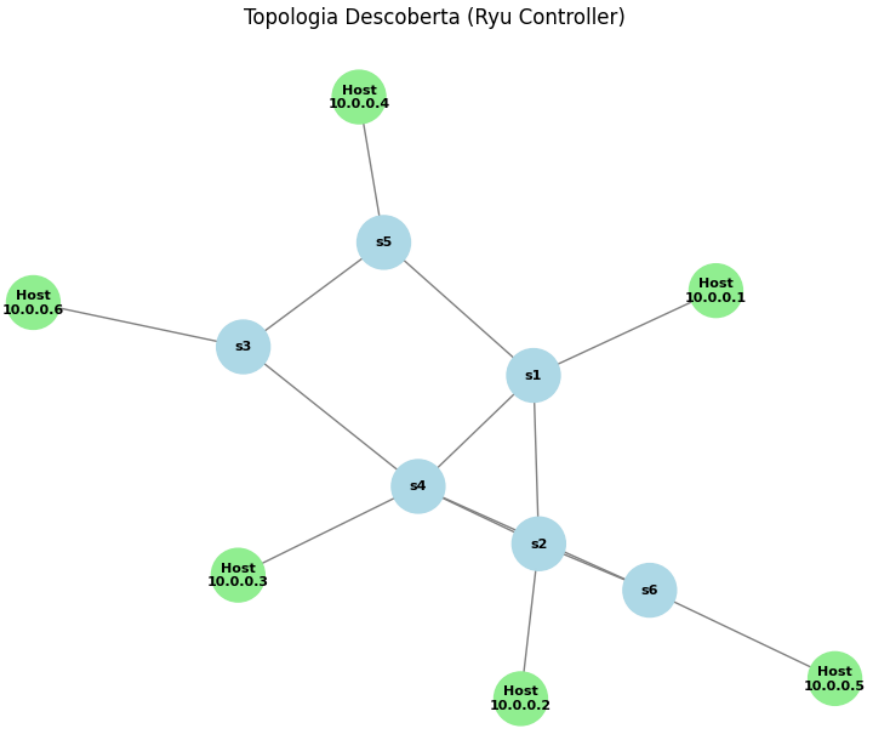


Figura 6: Grafo da topologia *Someloops* gerado com NetworkX a partir dos dados descobertos pelo Ryu.

Análise de Consistência (*Someloops*):

- Todos os 6 switches (`s1` a `s6`) e todas as 8 ligações bidirecionais entre eles (16 enlaces direcionais LLDP: `s1-s2`, `s1-s4`, `s1-s5`, `s2-s4`, `s2-s6`, `s3-s4`, `s3-s5`, `s4-s6`) foram fielmente mapeados pelo Ryu e reconstruídos no grafo do NetworkX.
- Observa-se um detalhe importante do Mininet: como os switches são instanciados conforme a leitura sequencial das linhas de `topo_someloops.txt`, a ordem de criação dos switches (e de chamada de `net.addHost()`) é `s1`, `s2`, `s4`, `s5`, `s6` e, por último, `s3` (conforme evidenciado na linha `h1 h2 h4 h5 h6 h3` da Figura 4). Como o Mininet atribui endereços IP sequencialmente na ordem de criação dos hosts, os IPs resultantes mapeados via Gratuitous ARP pelo Ryu refletem exatamente essa ordem (`s1: 10.0.0.1`, `s2: 10.0.0.2`, `s4: 10.0.0.3`, `s5: 10.0.0.4`, `s6: 10.0.0.5` e `s3: 10.0.0.6`).

4 Conclusão

A implementação cumpriu integralmente os objetivos propostos para a leitura, instanciação e descoberta dinâmica de topologias em ambientes SDN com OpenFlow 1.3. O parser desenvolvido para o Mininet permitiu transformar listas de arestas textuais arbitrárias em redes emuladas funcionais, enquanto o controlador Ryu demonstrou como combinar a descoberta ativa de enlaces em camada de enlace (via LLDP com `--observe-links` e a API `get_switch`/`get_link`) com a descoberta passiva de hosts via interceptação de pacotes *Gratuitous ARP* por mensagens `PacketIn`.

Por fim, a integração do controlador Ryu com a biblioteca NetworkX possibilitou validar visualmente a topologia descoberta em grafos com múltiplos caminhos e ciclos (*Triangle* e *Someloops*), preparando a base de código para futuros algoritmos de roteamento e engenharia de tráfego em SDN.

5 Referências de Arquivos no Repositório

Todos os códigos-fonte e imagens geradas encontram-se no repositório oficial do trabalho (https://github.com/vivendo88/Redes---Trabalho-SDN-1):

- `topologia.py`: Script de parsing da lista de arestas, instanciação da rede no Mininet e envio de *Gratuitous ARP*.
- `topo_discover.py`: Aplicação do controlador Ryu (OpenFlow 1.3) para descoberta de switches, enlaces e hosts, impressão no log e plotagem com NetworkX.
- `topo_triangle.txt` e `topo_someloops.txt`: Arquivos de definição das topologias de teste fixas.